



Lecture[06]

STRUCTURES IN C



Question?

- How to store the personal information of 50 students, including student ID number, name, gender, age, address, telephone number, and scores?
- How to store the bank account information of 100 customers, including account number, password, balance, date, and transaction records?

How many variables are required?

Is a multi-dimensional array sufficient?

Can we store the data with different types in a single variable?



Agenda

- Structure in C
- Keyword **struct**
- Structure definitions (定义)
- Structure declarations (声明)
- Structure initiations and assignments (初始化、赋值)
- Array of structure
- Operation ->
- typedef
- Working with function
- Self-referential Structure
- Example



Introduction

- Structures
 - Collections of related variables (aggregates) under one name
 - Can contain variables of different data types
 - E.g., student, bank account
 - Commonly used to define records to be stored in files
 - Combined with pointers, can create linked lists, stacks, queues, and trees

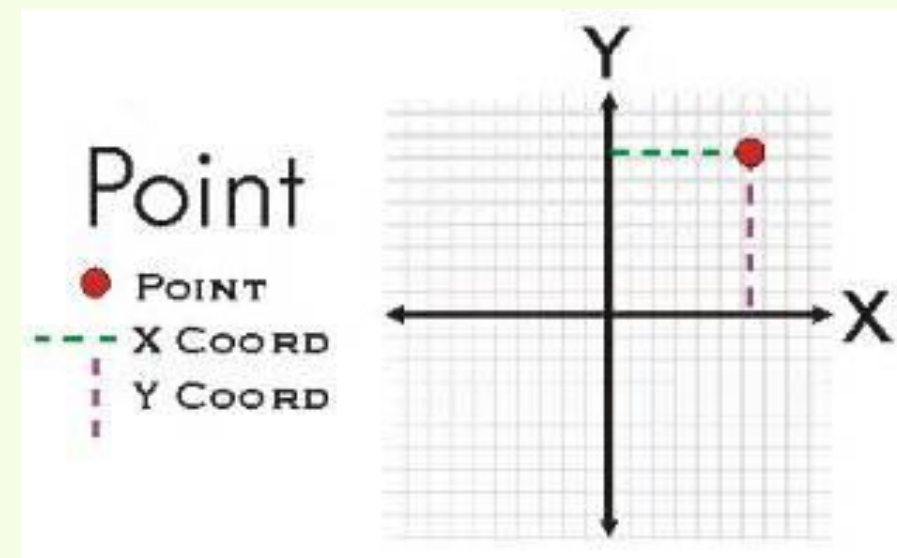


Structure Definitions

- Example

```
struct point {  
    int x  
    int y;  
};
```

- **struct** introduces the definition for structure point (x,y)
- **point** is the structure
- **point** contains two members (成员) of type **int**
 - x coordinate
 - y coordinate





Structure Definitions

- Example

```
struct card {  
    char *face;  
    char *suit;  
};
```

- **struct** introduces the definition for structure card
- **card** is the structure name and is used to declare variables of the structure type
- **card** contains two members of type **char ***
 - These members are **face** and **suit**



Structure Definitions

- **struct** information

—A **struct** cannot contain an instance (实例) of itself

```
struct point {  
    int x;  
    int y;  
    struct point b;  
};
```

← **This is wrong!**

Another example

```
struct bedroom  
{  
    struct bed b;  
    struct table t;  
    struct bedroom r;  
};
```

How many beds and tables are there in a bedroom?

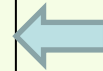
← **This is also wrong!**



Structure Definitions

- Can contain a member that is a pointer to the same structure type

```
struct point {  
    int x;  
    int y;  
    struct point* b;  
};
```



This is correct

- A structure definition does not reserve space in memory
 - It creates a new data type used to declare structure variables



Structure Declarations

- Declarations

- Declared like other variables:

- ```
struct card oneCard, deck[52], *cPtr;
```

- Can use a comma separated list:

- ```
struct card {  
    char *face;  
    char *suit;  
} oneCard, deck[ 52 ], *cPtr;
```



Structure Declarations

- Valid Operations （合法操作）
 - Assigning a structure to a structure of the same type
 - Taking the address (&) of a structure
 - Accessing the members of a structure
 - Using the **sizeof** operator to determine the size of a structure



Initializing Structures

- Initializer lists

- Example:

- ```
struct card oneCard = { "Three", "Hearts" };
```

- Assignment statements

- Example:

- ```
struct card threeHearts = oneCard;
```

- Could also declare and initialize **threeHearts** as follows:

- ```
struct card threeHearts;
```

- ```
threeHearts.face = "Three";
```

- ```
threeHearts.suit = "Hearts";
```



## Array of structures

- Give the size of the array in []
- Or we could use sizeof()
  - Compile-time unary operator
  - Use in #define (宏)

```
#define NTIMES (sizeof testTimes
/ sizeof(struct time))
```
  - Don't assume that the size of a structure is the sum of the sizes of its members.

```
struct X
{
 char a;
 int b;
 double c;
}S1;
```

```
sizeof(S1) = 16
sizeof(S1.a) = 1, sizeof(S1.b) = 4,
sizeof(S1.c) = 8
```

storage alignment

```
struct time
{
 int hour;
 int minutes;
 int seconds;
} testTimes[] =
{ { 11, 59, 59 },
 { 12, 0, 0 },
 { 1, 29, 59 },
 { 23, 59, 59 },
 { 19, 12, 27 } };
```



## Accessing Members of Structures

- Accessing structure members
  - Dot operator (.) used with structure variables

```
struct card myCard;
printf("%s", myCard.suit);
```

- Arrow operator (->) used with pointers to structure variables

```
struct DATA
{
 char key[10];
 char name[20];
 int age;
};
Struct DATA data;
Struct DATA *pdata;
data.age = 24;
pdata->age = 24;
```



## Operator ->

- Structure could be pointed

–Assign the pointer to structure

```
struct card *myCardPtr = &myCard;
```

–Access structure variable with (.)

```
(*myCardPtr).suit
```

–`myCardPtr->suit` is equivalent to

```
(*myCardPtr).suit
```



## Precedence (优先级) of (.) and ->

- They are at the top level (should be firstly considered)

–Before \* and ++/--

- Example:

```
Struct {
 int len;
 char *str;
} *p;
```

–++p->len, (++p)->len, p++->len

–\*p->str, \*p->str++, (\*p->str)++, \*p++->str

++p->len 先访问，再增加len，不是增加p

(++p)->len 在访问len之前先增加p

p++->len 先访问，再增加p

\*p->str 访问str所指向的东西

\*p->str++ 先访问str指向的东西，然后增加str

(\*p->str)++ 增加str所指向的东西

\*p++->str 先访问str指向的东西，然后增加p



## typedef (别名)

- **typedef**

- Creates synonyms (aliases) for previously defined data types
- Use **typedef** to create shorter type names
  - **typedef int Length**
  - **typedef char\* String**





## typedef

### –Example:

```
typedef struct card mycard;
struct card var;
mycard var;
```



```
typedef struct Card* CardPtr;
```

–Defines a new type name **CardPtr** as a synonym for type **struct Card \***

- -> everywhere

–**typedef** does not create a new data type

- Only creates an alias

```
typedef int myarray[100];
myarray var; // var是具有100个元素的int数组
```



## Using Structures With Functions

- Passing structures to functions
  - Pass entire structure
    - Or, pass individual members
  - Both pass **call by value**
- Example: Time structure
  - struct time{ ... }
  - struct time timeUpdate(struct time now)



## Using Structures With Functions

- To pass structures **call-by-reference**
  - Pass its address
  - Pass reference to it
  - `struct time *getTimebyIndex(int index, struct time *tab, int n)`
- Working with array of structures
  - `struct time testTimes[5]`

|              |   |          |    |
|--------------|---|----------|----|
| testTimes[0] | { | .hour    | 11 |
|              |   | .minutes | 59 |
|              |   | .seconds | 59 |
| testTimes[1] | { | .hour    | 12 |
|              |   | .minutes | 0  |
|              |   | .seconds | 0  |
| testTimes[2] | { | .hour    | 1  |
|              |   | .minutes | 29 |
|              |   | .seconds | 59 |
| testTimes[3] | { | .hour    | 23 |
|              |   | .minutes | 59 |
|              |   | .seconds | 59 |
| testTimes[4] | { | .hour    | 19 |
|              |   | .minutes | 12 |
|              |   | .seconds | 27 |



## Self-referential Structure (自引用结构)

- Self-referential structure is widely used to generate data structure
- Use of pointers to structures to create the links between objects
  - Use pointers to the same type of structure
  - Use pointers as members of the structure
- `typedef` and `struct`



## Summary on Structures in C

- Keyword **struct**:
  - Definition of a structure: `struct structName{}`;
  - Declaration of variable: `struct structName variableName`;
- A collection of related variables
- Structure and pointer
  - Structure can be pointed: `struct structname *structPointerName`;
  - members of a structure can be pointers (to outside of structure)
  - From pointer to a member of the structure it points to:  
`structePointerName->memberName`
- Work with function as an variable (call by value & call by reference)



## Example: High-Performance Card-shuffling and Dealing Simulation

- Pseudocode:
  - Create an array of card structures
  - Put cards in the deck
  - Shuffle the deck
  - Deal the cards



```
1 /* Fig. 10.3: fig10_03.c
2 The card shuffling and dealing program using structures */
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6
7 struct card {
8 const char *face;
9 const char *suit;
10 };
11
12 typedef struct card Card;
13
14 void fillDeck(Card * const, const char *[],
15 const char *[]);
16 void shuffle(Card * const);
17 void deal(const Card * const);
18
19 int main()
20 {
21 Card deck[52];
22 const char *face[] = { "Ace", "Deuce", "Three",
23 "Four", "Five",
24 "Six", "Seven", "Eight",
25 "Nine", "Ten",
26 "Jack", "Queen", "King" };
27 const char *suit[] = { "Hearts", "Diamonds",
28 "Clubs", "Spades" };
29
30 srand(time(NULL));
```



- 1. Load headers
- 1.1 Define struct
- 1.2 Function prototypes
- 1.3 Initialize deck[] and face[]
- 1.4 Initialize suit[]



```
31
32 fillDeck(deck, face, suit);
33 shuffle(deck);
34 deal(deck);
35 return 0;
36 }
37
38 void fillDeck(Card * const wDeck, const char * wFace[],
39 const char * wSuit[])
40 {
41 int i;
42
43 for (i = 0; i <= 51; i++) {
44 wDeck[i].face = wFace[i % 13];
45 wDeck[i].suit = wSuit[i / 13];
46 }
47 }
48
49 void shuffle(Card * const wDeck)
50 {
51 int i, j;
52 Card temp;
53
54 for (i = 0; i <= 51; i++) {
55 j = rand() % 52;
56 temp = wDeck[i];
57 wDeck[i] = wDeck[j];
58 wDeck[j] = temp;
59 }
60 }
```

Put all 52 cards in the deck.  
**face** and **suit** determined by  
remainder (modulus).

Select random number between 0 and 51.  
Swap element **i** with that element.



2. fillDeck

2.1 shuffle

2.2 deal

3. Function definitions

[Outline](#)





```
61
62 void deal(const Card * const wDeck)
63 {
64 int i;
65
66 for (i = 0; i <= 51; i++)
67 printf("%5s of %-8s%c", wDeck[i].face,
68 wDeck[i].suit,
69 (i + 1) % 2 ? '\t' : '\n');
70 }
```



Cycle through array and print  
out data.





Eight of Diamonds  
Eight of Clubs  
Seven of Hearts  
Ace of Clubs  
Deuce of Spades  
Seven of Spades  
Jack of Clubs  
King of Hearts  
Three of Hearts  
Three of Clubs  
Ten of Hearts  
Ten of Clubs  
Six of Clubs  
Six of Hearts  
Nine of Diamonds  
Jack of Spades  
King of Diamonds  
Nine of Spades  
Six of Spades  
Queen of Diamonds  
Ace of Spades  
King of Clubs  
King of Spades  
Queen of Hearts  
Four of Spades  
Four of Clubs

Ace of Hearts  
Five of Spades  
Deuce of Diamonds  
Ten of Diamonds  
Six of Diamonds  
Deuce of Clubs  
Ten of Spades  
Jack of Diamonds  
Three of Diamonds  
Nine of Clubs  
Deuce of Hearts  
Seven of Diamonds  
Queen of Spades  
Three of Spades  
Ace of Diamonds  
Five of Clubs  
Seven of Clubs  
Four of Hearts  
Eight of Spades  
Five of Diamonds  
Nine of Hearts  
Five of Hearts  
Four of Diamonds  
Eight of Hearts  
Jack of Hearts  
Queen of Clubs



Program Output

[Outline](#)